



FILE PATH TRAVERSAL

SIMPLE CASE

ACTIVIDAD 07

Contenido

File Path Traversal: <i>Simple Case</i>	2
Especificaciones del laboratorio.....	2
Objetivo del laboratorio	2
Marco Teórico	3
Definición y origen de la vulnerabilidad.....	3
Clasificación según OWASP Top 10 y triada CIA.....	3
Riesgos en un entorno real.....	3
Procedimiento	4
Explicación del Payload	7
Mitigación recomendada	8
Referencias.....	8

File Path Traversal: *Simple Case*

Especificaciones del laboratorio

Laboratorio: File Path Traversal, simple case.

Nivel: Apprentice

Tipo de explotación: File Path Traversal, lectura arbitraria de archivos

Objetivo del laboratorio

Exponer el contenido del archivo del sistema operativo */etc/passwd*, alojado fuera del directorio de recursos estáticos del servidor, manipulando el parámetro *filename* de la funcionalidad de carga de imágenes de una tienda en línea vulnerable. El objetivo académico es identificar, mediante interceptación de tráfico con Burp Suite, un punto donde la entrada del usuario se concatena sin validar a una ruta del sistema de archivos, y demostrar cómo esa falta de saneamiento permite escapar del directorio autorizado.

Marco Teórico

Definición y origen de la vulnerabilidad

El path traversal o directory traversal, es una vulnerabilidad del lado del servidor que permite a un atacante interactuar con archivos arbitrarios del sistema de archivos a los que no debería tener acceso, incluyendo código fuente de la aplicación, credenciales o archivos de configuración del sistema operativo. (PortSwigger, s.f)

Formalmente, este defecto está catalogado como **CWE-22: Improper Limitation of a Pathname to a Restricted Directory**, cuya definición establece que el producto construye una ruta de archivo a partir de una entrada externa sin neutralizar los elementos especiales que permitirían que esa ruta resuelva a una ubicación fuera del directorio restringido previsto (MITRE Corporation, 2024).

La causa de origen es que el backend recibe un valor controlador por el cliente, como lo es el parámetro de filename, y lo concatena directamente a una ruta del servidor en lugar de resolverlo contra una lista de recursos permitidos. La secuencia '../' es interpretada por el sistema operativo como una instrucción de navegación hacia el directorio padre, al repetirla varias veces, un atacante podría retroceder desde el subdirectorio de almacenamiento, hasta la raíz del sistema de archivos, y desde ahí construir la ruta hacia cualquier archivo legible por el proceso del servidor.

Clasificación según OWASP Top 10 y triada CIA

OWASP (Open Worldwide Application Security Project) es una fundación sin fines de lucro dedicada a la seguridad del software. Su Top 10 es el framework más usado en la industria para clasificar los riesgos más críticos y frecuentes en aplicaciones web, construido a partir del análisis de cientos de miles de aplicaciones reales.

Dentro del framework de OWASP, su Top 10 de 2025, esta debilidad se agrupa bajo la categoría **A01:2025 - Broken Access Control**, la de mayor incidencia entre todas las categorías evaluadas. El 100% de las aplicaciones analizadas presentó alguna forma de control de acceso roto (OWASP Foundation, 2025). El propio listado oficial de OWASP incluye explícitamente CWE-22 (Path Traversal) entre las debilidades mapeadas a esta categoría, confirmando que el fallo explotado en este laboratorio corresponde formalmente a un control de acceso roto ya que el servidor no verifica si el recurso solicitado pertenece al espacio autorizado para el usuario.

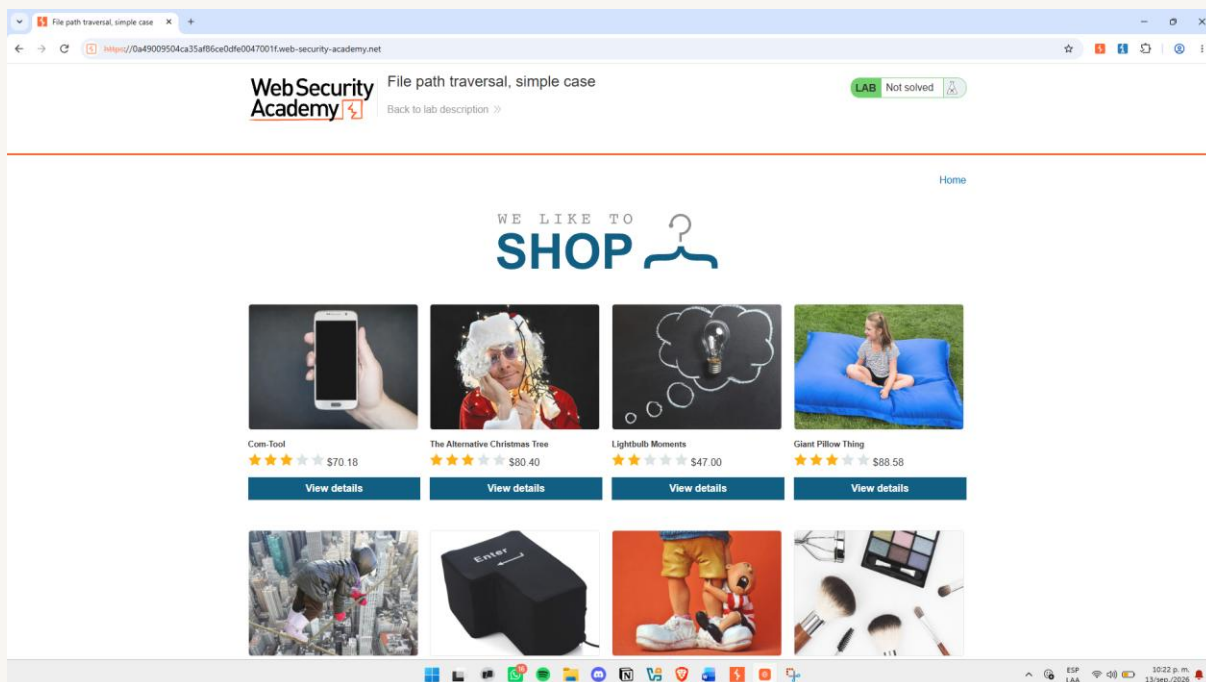
Evaluando el impacto bajo la triada de Confidencialidad, Integridad y Disponibilidad (CIA), el efecto inmediato recae sobre la Confidencialidad, ya que permite la exfiltración de archivos internos, credenciales o código fuente. Si el mismo endpoint permitiera además operaciones de escritura sobre rutas manipuladas, la Integridad y la Disponibilidad quedarían igualmente comprometidas, al poder sobrescribirse binarios, cron jobs o archivos de configuración críticos.

Riesgos en un entorno real

En un deploy real, esta vulnerabilidad se explota comúnmente para leer /etc/passwd o /etc/shadow, archivos .env con secretos, datos confidenciales de la infraestructura, o llaves privadas de SSH. Cuando además existe la capacidad de subir archivos o la posibilidad de sobrescribir scripts binarios, puede escalar hasta la Ejecución Remota de Código (RCE), lo que representa un escenario fatídico pues compromete totalmente el servidor.

Procedimiento

1. Se inició Burp Suite Community Edition, y se accedió al navegador Chromium integrado de la herramienta desde le botón *Open Browser*, el cual ya está enrutado para pasar todo su tráfico a través del proxy sin una configuración adicional.
2. Con ese navegador se accedió a la instancia del laboratorio de PortSwigger, donde se puede observar que la etiqueta del reto muestra el estado "Not solved" como punto de partida.



3. Al recorrer el catálogo de productos en el navegador de Burp, se revisó en paralelo la pestaña Proxy > HTTP history, donde queda registrado el historial completo de peticiones generadas por la página.

Ahí se observó que la carga de imágenes generaba automáticamente varias peticiones GET hacia el mismo endpoint, /image, cada una con un parámetro filename distinto (filename=6.jpg, filename=57.jpg, etc.). Este patrón indicó que el backend usa directamente ese valor para localizar y devolver un archivo del sistema, lo que hizo que fuera ideal para probar la manipulación de rutas.

#	Host	Meth...	URL	Params	Edited	Status code
1	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/			200
6	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=6.jpg	✓		
8	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=24.jpg	✓		
9	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=57.jpg	✓		
10	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/academyLabHeader			
11	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/			
12	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=61.jpg	✓		
13	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=30.jpg	✓		
14	https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net	GET	/image?filename=72.jpg	✓		

4. Se seleccionó una de las peticiones dirigidas a /image y se envió al módulo Repeater mediante el menú contextual (Send to Repeater), lo que permite editar el parámetro filename y reenviar la solicitud cuantas veces sea necesario sin depender de recargar el navegador en cada intento.



The screenshot shows the Burp Suite interface with a request and response view. The request is a GET to `/image?filename=6.jpg` with a status code of 200 and a length of 10477. The response is a 200 status code with a length of 10477. The response body contains the image data.

Request

```
1 GET /image?filename=6.jpg HTTP/2
2 Host: 0a49009504ca35af86ce0dfe0047001f.web-security-academy.net
3 Cookie: session=jyA2hsL7hptlFTNalCiwPthJfHwSNanr
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a49009504ca35af86ce0dfe0047001f.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: u=2, i
16
17
```

Response

```
1 200 /image?filename=6.jpg HTTP/2
2 Content-Type: image/jpeg
3 Content-Length: 10477
4
5 [Image Data]
```

5. En el valor del parámetro se construyó el payload `../../../../etc/passwd`, aprovechando que cada secuencia `../` retrocede un nivel en el directorio desde la carpeta base donde el servidor almacena las imágenes.

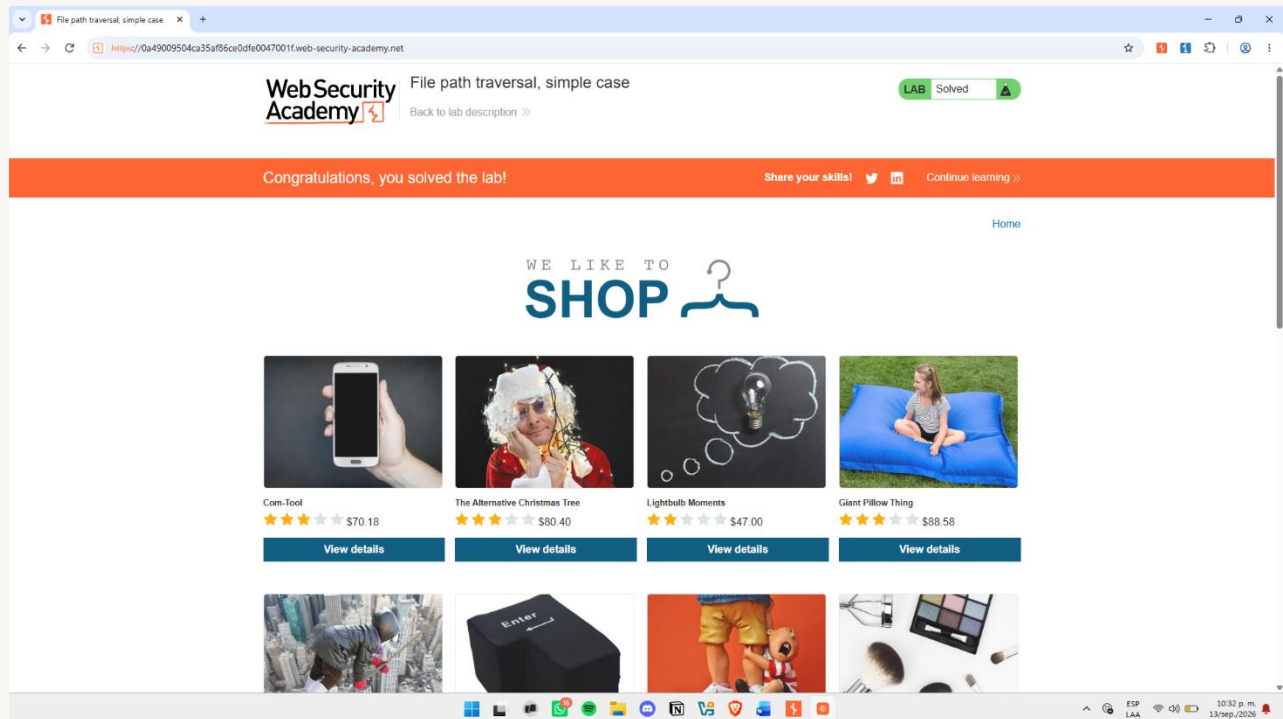


The screenshot shows the Burp Suite Repeater interface. The 'Request' tab is active, displaying a GET request to `/image?filename=../../../../etc/passwd HTTP/2`. The request body is highlighted with a red box. The 'Response' tab is also visible, showing the response from the target server.

6. Se reenvió la petición con este valor desde Repeater. El servidor respondió HTTP/2 200 OK, devolviendo en el cuerpo el contenido real de `/etc/passwd` del sistema operativo.

The screenshot shows the Burp Suite Repeater interface. The 'Response' tab is active, displaying the response from the target server. The response body is highlighted with a red box, showing the contents of `/etc/passwd`.

7. Finalmente, se regresó a la pestaña de la tienda y se refrescó la página. La plataforma de PortSwigger reconoció la explotación exitosa y actualizó el estado del laboratorio de "Not solved" a "Solved".



Explicación del Payload

El vector de ataque `../../etc/passwd` explota un error de diseño del backend, que es la ausencia de verificación sobre el resultado final de concatenar la carpeta base del servidor con el valor proporcionado por el cliente.

La secuencia `../` corresponde a una convención estándar de los sistemas de archivos UNIX para referenciar el directorio inmediatamente superior al directorio de trabajo actual. Al repetirse tres veces, el puntero de ruta retrocede sucesivamente desde el subdirectorio de almacenamiento de imágenes hasta alcanzar la raíz del sistema (`/`).

A partir de que se alcanza la raíz, el segmento `etc/passwd` completa la ruta hacia el archivo del sistema que almacena las cuentas de usuario locales, junto con sus identificadores, directorios de inicio y shells asignados por defecto.

La condición que habilita la explotación no es la sintaxis de la secuencia en sí, que es una funcionalidad normal del sistema operativo, sino que la ausencia de una comprobación posterior a la concatenación. El backend nunca evalúa si la ruta resultante permanecía dentro del directorio de imágenes autorizado antes de proceder a leer el archivo correspondiente y devolver su contenido en la respuesta.

De manera que al no existir una sanitización la función concatena la entrada y resuelve la ruta, forzando al endpoint a leer y devolver el archivo.

Al no existir sanitización ni validación contra una whitelist de caracteres o rutas permitidas, la función del backend concatena la entrada de forma cruda (por ejemplo, `'/var/www/images/' + '../../etc/passwd'`), resolviendo la ruta canónica en `'/etc/passwd'` y forzando a la API del sistema de archivos a leer y devolver un archivo sensible del sistema operativo.

Mitigación recomendada

Las medidas de mitigación pueden agruparse en dos niveles: aquellas que eliminan la causa estructural de la vulnerabilidad, y aquellas que actúan como controles de contención en caso de que las primeras fallen.

En el primer nivel, la medida más efectiva consiste en eliminar por completo la dependencia de la entrada del cliente para construir rutas de archivo. En lugar de recibir un nombre de archivo, el sistema puede recibir un identificador asociado a la ruta real mediante una tabla en base de datos, de forma que la resolución de la ruta ocurra íntegramente en el servidor y sea independiente del valor enviado por el cliente. Cuando el manejo de un nombre de archivo resulte indispensable, debe aplicarse una validación estricta por whitelist, aceptando exclusivamente caracteres alfanuméricos y extensiones predefinidas, y rechazando de forma explícita cualquier separador de ruta o secuencia de puntos antes de procesar la solicitud.

En el segundo nivel, puede emplearse la canonicalización de la ruta solicitada con funciones nativas del lenguaje de desarrollo, verificando que la ruta absoluta obtenida se encuentre contenida dentro del directorio base autorizado.

En el plano de infraestructura, se pueden restringir los privilegios bajo los cuales se ejecuta el proceso del servidor, con un entorno aislado o una cuenta de servicio con permisos mínimos, limita el impacto de una posible falla en la validación previa, dado que el propio sistema operativo impediría el acceso a directorios críticos como /etc o /root. También la implementación de un Web Application Firewall permite detectar patrones de evasión de rutas antes de que alcancen la lógica de la aplicación, mientras que la incorporación de análisis estático de código y pruebas de penetración periódicas fortalecen la identificación temprana de este tipo de debilidades a lo largo del ciclo de desarrollo

Referencias

MITRE Corporation. (2024). *CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal')*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>

OWASP Foundation. (2025). *A01:2025 – Broken Access Control*. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

PortSwigger. (s.f.). *What is path traversal, and how to prevent it?* Web Security Academy. <https://portswigger.net/web-security/file-path-traversal>